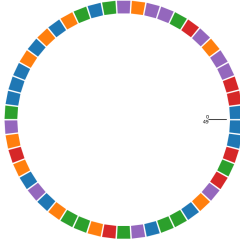


Discrete Bayes Filter for Circular Robot Localization

Description

A toy boat randomly moves about the edge of a circularly looped pool with 50 colored tiles on the floor. There are 5 different colors.



The only sensor the toy has detects the color of the tile immediately beneath the boat. Because of the water, the sensor does not always detect the correct color. The toy boat also has a map of the colored tiles with which to compare the sensor readings. The wind tends to move the boat around the pool in one direction more than another. With knowledge of the map, the sensor readings, the PMF of the sensor, and the PMF of the dynamics, the boat can figure out where it is in the pool.

Bayes Filter

The robot's position is estimated using a discrete Bayes filter. The belief over the state, represented as a probability mass function $p(x)$, is updated recursively through two steps:

Measurement Update: The prior belief is updated using the likelihood of the observed measurement:

$$p(x) \propto p(z|x)p(x)$$

Time Propagation: The updated belief is propagated forward using the Markov transition matrix:

$$p(x_{k+1}) = Mp(x_k)$$

Performance Metrics

Three metrics are used to evaluate the filter:

Accuracy: The fraction of time steps where the estimated position, given by the maximum of $p(x)$, matches the true position.

Validity: A measure of how well the probability distribution represents the true state. When the estimate matches the truth, the contribution is weighted by the inverse of the estimate probability. A valid estimator produces a value close to 1.

Goodness: The fraction of time steps where the maximum probability exceeds a threshold of 0.5, indicating high confidence in the estimate.

Monte Carlo Simulation

To evaluate the performance of the estimator, all experiments are conducted using Monte Carlo simulations with 100 independent runs. For each run, a new trajectory and set of measurements are generated, and the resulting accuracy, validity, and goodness metrics averaged across all runs.

Nominal Case

The filter is first evaluated using matched parameters for both data generation and estimation, with sensor reliability $p_c = 0.8$ and motion model $p_m = [0.2, 0.3, 0.5]$. This case serves as a baseline and is expected to produce a validity value close to 1, indicating that the estimated probability mass function accurately represents the true system behavior.

Sensor Model Mismatch

To study the effect of model mismatch, the true sensor reliability is fixed at $p_c = 0.8$, while the estimator assumes different values.

First, the estimator assumes $p_c = 0.6$, underestimating the sensor accuracy. Next, the estimator assumes $p_c = 0.99$, overestimating the sensor accuracy.

Parameter Sweep

A broader analysis is conducted by varying both the sensor reliability and motion model. Four motion models are considered:

Set	p_m
1	[0.3, 0.3, 0.4]
2	[0.2, 0.2, 0.6]
3	[0.1, 0.1, 0.8]
4	[0.1, 0.8, 0.1]

For each motion model, the sensor reliability p_c is varied from 0.6 to 0.95 in increments of 0.05. This results in 32 total scenarios. For each scenario, the accuracy metric is computed as the average over 100 Monte Carlo runs.

The resulting accuracy curves illustrate how estimation performance depends on both sensor quality and system dynamics.

Results

Nominal Case

Figure 1 shows the results when both generation and estimation use the same p_c value.

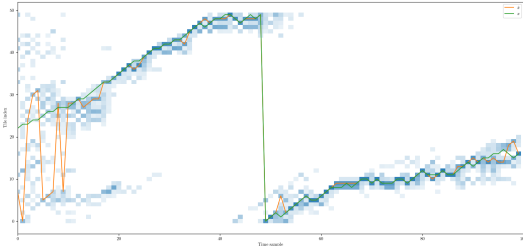


Figure 1: Validity: 1.001 Goodness: 0.59 Accuracy: 0.5522

Sensor Model Mismatch

Figure 2 shows the results when only estimation uses a p_c value of 0.6.

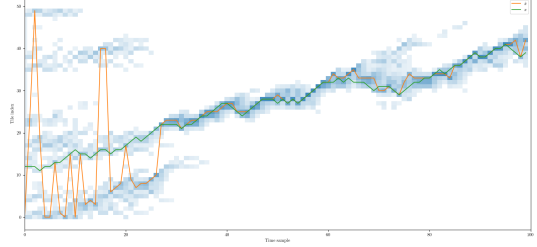


Figure 2: Validity: 1.3344 Goodness: 0.31 Accuracy: 0.5413

The validity and goodness scores differed significantly in this scenario. Since the p_c value was less for our estimator than reality, the estimator was underconfident in our sensor. This led to a validity higher than 1 because our estimator believed our sensor to be less accurate than it truly was. This also decreased our goodness score because our peak probabilities were lower due to the decreased belief in the accuracy of our sensor.

Figure 3 shows the results when only estimation uses a p_c value of 0.99.

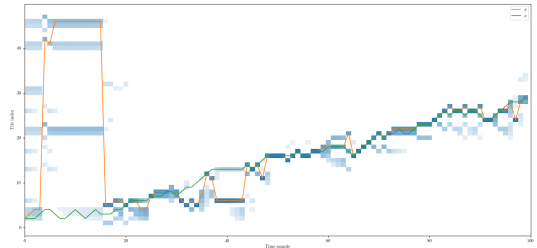


Figure 3: Validity: 0.7776 Goodness: 0.76 Accuracy: 0.5413

The validity and goodness scores again differed in this scenario. Since the p_c value was greater for our estimator than reality, the estimator was overconfident in our sensor. This led to a validity lower than 1 because our estimator believed our sensor to be more accurate than it truly was. This also increased our goodness score because our peak proba-

bilities were higher due to the increased belief in the accuracy of our sensor.

Parameter Sweep

Figure 4 shows, in a single figure, the **accuracy** scores for each of the **pm** sets, labeling each curve “Set {number}”.

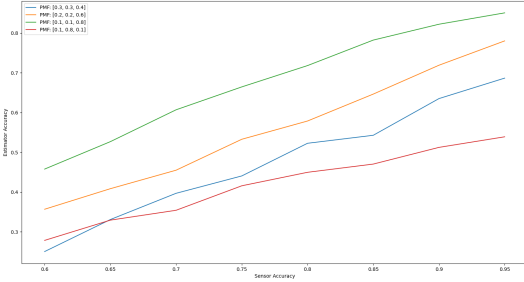


Figure 4: Accuracies of each motion model over 100 Monte Carlo runs

The first observation worth noting is the most obvious and self-explanatory. As the sensor accuracy increases, the estimator accuracy increases. This makes sense since a good estimator would be more confident in higher accuracy measurements.

The second observation of importance is the general rule of increased estimator accuracy for larger dynamics probabilities. What I mean by this is that the most accurate estimator was the one for the dynamics PMF: $[0.1, 0.1, 0.8]$. Compared to the estimator for the dynamics PMF: $[0.3, 0.3, 0.4]$, the accuracy was much greater for the one with the greatest single probability in the PMF. This makes sense because if the dynamics say the robot will move left with probability 0.8, the estimator will become very confident that the robot will continue to move forward. However, if the robot moves closer to a uniform distribution, it is much harder for the estimator to predict where the robot will go next, leading to a lower estimator accuracy.

The last major observation I noticed was quite un-intuitive. The outlier for the rule I just mentioned above was the PMF: $[0.1, 0.8, 0.1]$. Even though it had the highest single probability, the estimator accuracy was actually the lowest compared to the other PMFs. This is mainly due to the fact that the dynamics of this robot mean it will most likely stay put, compared to the robot of $[0.1, 0.1, 0.8]$ being most likely to move right (and explore). This hurts

the estimators accuracy for two reasons: The first being that each step provides very little new information when the robot is mostly dormant. If the robot is staying put over many steps, the filter will only become slightly more confident in its estimation. The second reason is that the filter will have significantly more trouble recovering from errors. If the dynamics say the robot is much more likely to stay stationary and the measurement has lead our estimator to believe we are in the wrong position, our estimator may be stuck in that position for a longer period of time. Whereas the estimator for the dynamics of the exploring robot will get new measurements at a much greater frequency, giving the filter more information to process and make a good estimate.

Discussion

The results demonstrate that accurate localization depends on both the quality of the sensor model and dynamics of the system. When the assumed sensor model matches the true system, the filter produces valid probability estimates. However, mismatches in the assumed sensor reliability lead to degrade performance, particularly in the validity metric.

Higher sensor accuracy generally improves estimation performance, while motion models with stronger directional bias lead to more predictable and accurate localization. These results highlight the importance of correctly modeling both system dynamics and measurement uncertainty in Bayesian estimation.